

Common Stuff between Tests

To make the other 9 tests **structurally correct** (i.e., they compile, dispatch correctly from `tb_top`, interact with BFM, drive coverage/ref-model, and obey the grading log contract), you should copy the **same “shape”** as `sanity_test.sv`.

Below is the **common syntax/structure contract** your tests should follow.

1) Each test file must have an include guard

So it won't compile twice if included/compiled in multiple places.

Example pattern (match your sanity test style):

```
`ifndef REG_ACCESS_TEST_SV
`define REG_ACCESS_TEST_SV
// ... file content ...
`endif
```

Rule: macro name should be unique per file.

2) Each test should be a class with a static `run()` task

Your dispatcher pattern (from the scaffold) is based on calling:

```
<test_class_name>::run(ref_model, coverage);
```

So every test must follow:

```
class reg_access_test;
    static task run(ref spi_ref_model ref_model,
                   ref spi_coverage_col coverage);
        // ...
    endtask
endclass
```

Why this matters

- SV cannot pass modules as **ref** arguments, so the BFM's are accessed via **hierarchical references** (`tb_top.u_apb_bfm ...`), while the ref model and coverage are passed as class handles.
-

3) Use the same ref argument types and order

Match exactly what `tb_top` expects to pass.

Your sanity test uses:

```
static task run(ref spi_ref_model    ref_model,  
               ref spi_coverage_col coverage);
```

All tests should use exactly this signature, unless you also update `tb_top` (not recommended).

4) Use the same APB register address localparams (or include a shared header)

Right now `sanity_test.sv` repeats these localparams:

```
localparam [7:0] APB_CTRL    = 8'h00;  
...  
localparam [7:0] APB_DELAY   = 8'h20;
```

Two acceptable approaches (pick one and be consistent):

A) Repeat the same localparams in every test (simple, but duplicated).

B) Put them in one shared header (recommended long-term), e.g.

- `tb/apb_regs.svh`
and then in every test:

```
`include "apb_regs.svh"
```

If you do B, keep the include path in Makefile (`+incdir+./tb` etc.).

5) Access BFM's via hierarchical references (same naming)

Your tests assume these exist in `tb_top` :

- `tb_top.u_apb_bfm.apb_write(addr, data)`
- `tb_top.u_apb_bfm.apb_read(addr, rd)`
- `tb_top.bfm_mode`
- `tb_top.bfm_pattern`

So your other tests must also use the same handles.

Structural requirement: `tb_top` must instantiate BFM's with those exact instance names and expose those control variables as `tb_top.<name>` .

If a test needs additional control knobs (e.g., another SPI pattern), add them in `tb_top` once, and document them.

6) Print consistent informational messages (optional but useful)

Not required by grader, but helps debugging.

Pattern:

```
$display("[INFO] <test>: starting");  
...  
$display("[INFO] <test>: finished, errors=%0d", ref_model.error_count);
```

7) Never print `[TEST_PASSED]` / `[TEST_FAILED]` inside the test

In your scaffold, `tb_top` should be responsible for printing exactly one final line.

So inside tests:

- do checks
- increase `ref_model.error_count` or call `ref_model.error( ... )`
- return to `tb_top`
- `tb_top` prints pass/fail exactly once

If each test prints pass/fail too, you risk multiple lines and grading failure.

8) Every test must be deterministic w.r.t **+SEED**

Even if you add randomization, you must seed it from the same plusarg seed used globally.

Common pattern (usually done in **tb_top** once):

- read **+SEED=<n>**
- call **std::randomize** with **srandom(seed)** or seed your RNG objects

Then tests should not override it randomly.

9) Timeouts / bounded loops are mandatory

Your sanity test uses:

```
repeat (500) begin
  apb_read(STATUS, ... )
  if (!BUSY) break;
end
```

Every test that waits for something must have:

- **repeat(N)** loops
- or a time-based timeout
and if timeout happens, call an error:

```
ref_model.error("timeout waiting for BUSY=0");
```

(or **\$error("[CHECKER_ERROR] ... ")**)

This prevents infinite hangs (which become test timeouts → failures).

10) Use the reference model + coverage in a consistent way

Your sanity test follows a good pattern:

1. Configure BFM (SPI mode/pattern)
2. Program DUT registers via APB
3. Tell coverage about configuration:

```
coverage.sample_config( ... );
```

4. Tell ref_model what should happen:

```
ref_model.predict_single_byte( ... );
```

5. Drive stimulus

6. Read results and check:

```
ref_model.check_rx(rd);
```

For other tests, repeat the same conceptual flow:

- **predict → stimulate → observe → check**
- and sample coverage at meaningful events:
 - config chosen
 - transfer done
 - interrupt occurred
 - FIFO level reached

11) Naming must match grading-required test names

Even though your file may be `tests/mode_coverage_test.sv`, what matters is:

- the dispatcher recognizes `"mode_coverage_test"`
- the class is named `mode_coverage_test`
- `Makefile: REGRESSION_TESTS` includes `mode_coverage_test`

So for all required tests, keep **consistent triple**:

- **filename:** `tests/<name>.sv`
- **class:** `class <name>;`
- **dispatcher case string:** `"<name>"`

Required list (from grading md you posted):

- `sanity_test`
- `reg_access_test`
- `mode_coverage_test`

- `width_coverage_test`
 - `fifo_stress_test`
 - `interrupt_test`
 - `clk_div_corner_test`
 - `loopback_test`
 - `delay_transfer_test`
 - `error_injection_test`
-

12) Common “skeleton” you can copy for every test

```
`ifndef REG_ACCESS_TEST_SV
`define REG_ACCESS_TEST_SV

// (either define localparams here OR `include "apb_regs.svh")

class reg_access_test;

    static task run(ref spi_ref_model ref_model,
                   ref spi_coverage_col coverage);

        bit [31:0] rd;

        $display("[INFO] reg_access_test: starting");

        // 1) Optional: configure SPI BFM knobs if needed
        // tb_top.bfm_mode = ...
        // tb_top.bfm_pattern = ...

        // 2) Program DUT regs via APB
        // tb_top.u_apb_bfm.apb_write( ... )

        // 3) Predict expected behavior
        // ref_model.predict_...

        // 4) Drive stimulus + wait with timeout
        // repeat(N) ...

        // 5) Read results and check
        // tb_top.u_apb_bfm.apb_read( ... )
        // ref_model.check_...

        // 6) Sample coverage
```

```
// coverage.sample_ ...

    $display("[INFO] reg_access_test: finished, errors=%0d",
ref_model.error_count);
    endtask

endclass

`endif
```

Quick checklist for your team (structural “musts”)

For each of the 9 remaining tests, verify:

1. `ifndef/define/endif` guard exists
2. `class <testname>;` exists (name matches required name)
3. `static task run(ref spi_ref_model, ref spi_coverage_col);` exists
4. Uses `tb_top.u_apb_bfm.apb_write/read` (no direct APB pins)
5. Uses bounded waits / timeouts
6. Does not print `[TEST_PASSED]` / `[TEST_FAILED]`
7. Updates `ref_model.error_count` only through `ref_model` or checker utilities
8. Samples coverage via `coverage` object
9. No hardcoded DUT file paths, no reliance on golden RTL paths in SV

If you want, paste your repo's `tb/tb_top.sv` and `env/ref_model.sv` headers (top ~50 lines each). I can then tell you **exactly** what methods (like `predict_single_byte`, `check_rx`, `sample_config`) already exist and what common helper APIs you should standardize across all tests to keep the implementation simple for beginners.